

1 var vs let vs const?

Ans: They differ in scope, hoisting, and mutability.

	var	let	const
Scope	Function	Block	Block
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No
Hoisting	Yes (init as undefined)	Yes (TDZ)	Yes (TDZ)
Typical use	Legacy code	Reassignable values	Constants

TDZ = Temporal Dead Zone — the variable exists but cannot be accessed before its declaration line.

2 What is hoisting?

Ans: Hoisting is JavaScript's behavior of moving declarations to the top of their scope before code runs.

- **var** → hoisted and initialized as undefined
- **let / const** → hoisted but not initialized (TDZ)
- **Functions** → fully hoisted (including the body)

```
console.log(a); // undefined
var a = 10;

console.log(b); // ReferenceError
let b = 20;
```

3 What is scope?

Ans: Scope determines where a variable is accessible from.

- **Global scope** — accessible everywhere
- **Function scope** — accessible inside the function
- **Block scope** — accessible inside { } (applies to let/const)

```
let x = 10; // global
function test() {
  let y = 20; // function scope
  if (true) {
    let z = 30; // block scope
  }
}
console.log(x); // 10
```

4 What are closures?

Ans: A closure is a function that "remembers" variables from its outer (lexical) scope even after the outer function has finished executing.

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    return count;
  };
}
const fn = outer();
fn(); // 1
fn(); // 2
```

Closures are the mechanism behind data privacy, memoization, and currying in JS.

5 What is lexical scope?

Ans: Lexical scope means a variable's scope is determined by where it is physically written in the source code. Inner functions have access to variables of their outer functions.

```
function outer() {
  let a = 10;
  function inner() {
    console.log(a); // 10
  }
  inner();
}
```

6 What is the Event Loop?

Ans: The Event Loop lets JavaScript perform non-blocking operations despite being single-threaded.

1. The Call Stack executes synchronous code.
2. Web APIs handle asynchronous tasks (timers, fetch, DOM events).
3. The Callback / Task Queue holds completed async callbacks.
4. The Event Loop moves tasks from the queue to the stack once it's empty.

Flow: Call Stack → Event Loop → Callback Queue, fed by Web APIs.

7 What is the Call Stack?

Ans: A data structure that tracks function calls, following LIFO (Last In, First Out) order.

- When a function is called, it's pushed onto the stack.
- When it returns, it's popped off the stack.
- If the stack overflows → "Maximum call stack size exceeded" error.

8 What is a callback?

Ans: A callback is a function passed as an argument to another function, to be executed later.

```
function greet(name, cb) {
  console.log("Hello " + name);
  cb();
}
greet("Shubh", function () {
  console.log("Callback executed");
});
```

Commonly used in async operations: setTimeout, event handlers, API calls.

9 What is a Promise?

Ans: A Promise is an object representing the eventual completion (or failure) of an asynchronous operation.

- **States:** Pending, Fulfilled, Rejected
- **Methods:** .then(), .catch(), .finally()

```
let promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) resolve("Done");
  else reject("Error");
});
promise
  .then(res => console.log(res))
  .catch(err => console.log(err));
```

10 What is async/await?

Ans: Syntactic sugar over Promises that lets asynchronous code be written in a synchronous-looking style.

- **async** → makes a function always return a Promise.
- **await** → pauses execution until the Promise settles.

```
async function getData() {
  try {
    let res = await fetch("data.json");
    let data = await res.json();
    console.log(data);
  } catch (err) {
    console.log(err);
  }
}
getData();
```

11 What are object methods?

Ans: Methods are functions stored as object properties; they define an object's behavior.

```
const user = {
  name: "Shubh",
  greet: function () {
    return "Hello " + this.name;
  }
};
console.log(user.greet()); // Hello Shubh
```

12 What is a prototype?

Ans: A prototype is an object from which other objects inherit properties and methods. Every object has an internal link to its prototype.

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function () {
  return "Hi " + this.name;
};
const p1 = new Person("Shubh");
console.log(p1.greet()); // Hi Shubh
```

13 What is the prototype chain?

Ans: When an object tries to access a property, JS first looks at its own properties, then its prototype, and so on up the chain until it reaches null.

```
p1 → Person.prototype → Object.prototype → null
(object)                (root object)
```

14 What is the "this" keyword?

Ans: this refers to the object that is executing the current function — its value depends on how the function is called.

Context	"this" refers to
In a method	the owning object
In a plain function	global object (undefined in strict mode)
In an arrow function	lexical — inherited from parent scope
In a constructor	the new instance

```
const obj = {
  name: "Shubh",
  sayHi() { console.log(this.name); }
};
obj.sayHi(); // Shubh
const fn = obj.sayHi;
fn(); // undefined (strict mode)
```

15 What does bind() do?

Ans: bind() returns a new function with this permanently set to the given value — it does not call the function immediately.

```
const obj = { name: "Shubh" };
function sayHi() { console.log("Hi " + this.name); }
const bound = sayHi.bind(obj);
bound(); // Hi Shubh
```

16 What does call() do?

Ans: call() invokes the function immediately, with this set to the provided value. Arguments are passed individually.

```
function add(a, b) {
  console.log(this.x + a + b);
}
add.call({ x: 10 }, 5, 3); // 18
```

17 What does apply() do?

Ans: Same as call(), but arguments are passed as an array.

```
function add(a, b) {
  console.log(this.x + a + b);
}
add.apply({ x: 10 }, [5, 3]); // 18
```

18 What is a constructor function?

Ans: A function used with the new keyword to create objects with shared structure.

```
function Car(brand) {
  this.brand = brand;
  this.start = function () {
    return this.brand + " started";
  };
}
const c = new Car("Tesla");
console.log(c.start()); // Tesla started
```

19 What are classes?

Ans: Classes are syntactic sugar over constructor functions, introduced in ES6.

```
class Car {
  constructor(brand) { this.brand = brand; }
  start() { return this.brand + " started"; }
}
const c = new Car("BMW");
console.log(c.start()); // BMW started
```

20 What is inheritance?

Ans: Inheritance lets a class inherit properties and methods from another class using extends.

```
class Animal {
  speak() { return "Animal speaks"; }
}
class Dog extends Animal {
  speak() { return "Dog barks"; }
}
const d = new Dog();
console.log(d.speak()); // Dog barks
```

Dog inherits speak() from Animal and overrides it here.

21) What does map() do?

Ans: Creates a new array by applying a function to every element.

```
const nums = [1, 2, 3, 4];
const double = nums.map(n => n * 2);
// [2, 4, 6, 8]
```

22) What does filter() do?

Ans: Creates a new array containing only the elements that pass a test (return true).

```
const nums = [1, 2, 3, 4, 5, 6];
const even = nums.filter(n => n % 2 === 0);
// [2, 4, 6]
```

23) What does reduce() do?

Ans: Reduces the array to a single accumulated value.

```
const nums = [1, 2, 3, 4];
const sum = nums.reduce((acc, cur) => acc + cur, 0);
// 10
```

Syntax: `arr.reduce(callback(acc, cur, index, arr), initialValue)`

24) What does forEach() do?

Ans: Executes a function for each element. It does not return a new array — used purely for side effects.

```
const nums = [1, 2, 3];
nums.forEach(n => console.log(n)); // 1 2 3
```

Good for side effects: logging, API calls, DOM updates.

25) What do some() and every() do?

Ans: `some()` returns true if at least one element satisfies the condition; `every()` returns true only if all elements do.

```
const nums = [2, 4, 6, 8];
nums.some(n => n > 6); // true
nums.every(n => n % 2 === 0); // true
```

26) What does find() do?

Ans: Returns the first element that satisfies the condition (or undefined if none found).

```
const users = [{id:1,name:"A"},{id:2,name:"B"}];
const user = users.find(u => u.id === 2);
// { id: 2, name: "B" }
```

27) What does findIndex() do?

Ans: Returns the index of the first element satisfying the condition (or -1 if none).

```
const nums = [10, 20, 30, 40];
const idx = nums.findIndex(n => n === 30);
// 2
```

28) What does splice() do?

Ans: Mutates the original array by adding, removing, or replacing elements.

Syntax: `arr.splice(start, deleteCount, item1, item2, ...)`

```
const arr = [1, 2, 3, 4, 5];
arr.splice(1, 2, 'a', 'b');
// [1, 'a', 'b', 4, 5]
```

29) What does slice() do?

Ans: Returns a shallow copy of a portion of an array — it does not mutate the original.

Syntax: `arr.slice(start, end)` — end is not included.

```
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 4); // [2, 3, 4]
arr.slice(2); // [3, 4, 5]
```

30) What is debouncing?

Ans: Debouncing delays a function's execution until a specified wait time has passed since the last time it was invoked.

Use cases: search input, window resize, button click.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
const debouncedFn = debounce(() => {
  console.log("API call");
}, 500);
```

31) What is throttling?

Ans: Throttling ensures a function runs at most once within a given time interval, regardless of how many times it's triggered.

Use cases: scroll, resize, mouse move, API rate limiting.

```
function throttle(fn, delay) {
  let last = 0;
  return function (...args) {
    let now = Date.now();
    if (now - last >= delay) {
      last = now;
      fn.apply(this, args);
    }
  };
}
```

32) What is currying?

Ans: Currying converts a function with multiple arguments into a sequence of functions, each taking a single argument.

```
function add(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
}
console.log(add(1)(2)(3)); // 6
```

33) What is memoization?

Ans: An optimization technique that caches results of expensive function calls, returning the cached result for repeated inputs.

```
function memoize(fn) {
  const cache = {};
  return function (n) {
    if (cache[n]) return cache[n];
    const result = fn(n);
    cache[n] = result;
    return result;
  };
}
const square = memoize(n => n * n);
square(5); // calculated
square(5); // from cache
```

Great for recursive or heavy calculations.

34) Deep copy vs shallow copy?

Feature	Shallow Copy	Deep Copy
Definition	Copies top-level properties only	Copies all nested properties
Methods	Object.assign(), spread {...}, slice()	JSON.parse/stringify, structuredClone(), custom recursion
Nested refs	Shared with original	Independent, new reference
Performance	Faster	Slower

35) What is the spread operator (...)?

Ans: Expands an iterable (array, string, object) into individual elements.

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // {a:1,b:2,c:3}
```

36) What is the rest operator (...)?

Ans: Collects multiple elements into a single array or object — the opposite of spread.

```
function sum(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}
sum(1, 2, 3, 4); // 10

const { a, ...others } = { a: 1, b: 2, c: 3 };
// others = { b: 2, c: 3 }
```

37) What is destructuring?

Ans: A syntax for unpacking values from arrays or properties from objects into variables.

```
// Array
const [a, b, ...rest] = [1, 2, 3, 4]; // a=1,b=2,rest=[3,4]

// Object
const { name, age, ...info } = {
  name: "Shubh", age: 25, city: "Delhi", role: "Dev"
};
```

38) What is optional chaining (?.)?

Ans: Allows safe access to nested properties without checking each level manually.

```
const user = { name: "Shubh", address: { city: "Delhi" } };
console.log(user.address?.city); // Delhi
console.log(user.contact?.mobile); // undefined, no error
```

Prevents "Cannot read property of undefined" errors.

39) How do you deep-copy an object?

Ans: Use structuredClone() (modern browsers) or JSON serialization.

```
const original = {
  name: "Shubh",
  hobbies: ["coding", "music"],
  address: { city: "Delhi" }
};
const copy1 = structuredClone(original);
const copy2 = JSON.parse(JSON.stringify(original));
```

40) How do you shallow-copy an object?

Ans: Use the spread operator or Object.assign().

```
const original = { a: 1, nested: { b: 2 } };
const copy = { ...original };
copy.a = 10; // independent
copy.nested.b = 20; // affects original (shared ref)
```

Nested objects are still shared by reference in a shallow copy.

Q41 What is the DOM?

Ans: The Document Object Model is a programming interface for HTML documents — it represents the page as a tree of nodes.

Common methods: `getElementById()`, `querySelector()`, `createElement()`, `appendChild()`, `removeChild()`.

Q42 What is the BOM?

Ans: The Browser Object Model represents objects provided by the browser to work with everything outside the document.

Object	Use
<code>window</code>	Global object, browser window
<code>navigator</code>	Info about browser & OS
<code>location</code>	URL related info & navigation
<code>history</code>	Browser history (back, forward)
<code>screen</code>	Screen related info

Q43 What is event delegation?

Ans: A technique where a single event listener on a parent element handles events for its children — useful for dynamically added elements.

```
document.getElementById('parent')
  .addEventListener('click', function (e) {
    if (e.target && e.target.matches('li')) {
      console.log('li clicked:', e.target.innerText);
    }
  });
```

Q44 What is event bubbling?

Ans: The propagation of an event from the target element up through its ancestors (Target → Parent → ... → document).

```
document.querySelector('div')
  .addEventListener('click', () => {
    console.log('div clicked');
  }, false); // false = bubbling (default)
```

Q45 What is event capturing?

Ans: The propagation of an event from the root (document) down to the target element (document → ... → Parent → Target).

```
document.querySelector('div')
  .addEventListener('click', () => {
    console.log('div clicked');
  }, true); // true = capturing
```

Q46 What is localStorage?

Ans: Stores data with no expiration date — data persists even after the browser is closed.

```
localStorage.setItem('name', 'Shubh');
let n = localStorage.getItem('name');
localStorage.removeItem('name');
localStorage.clear();
```

Stores only strings. Capacity ~5-10MB per domain.

Q47 What is sessionStorage?

Ans: Stores data for a single tab session — data is cleared when the tab is closed.

```
sessionStorage.setItem('token', 'abc123');
let t = sessionStorage.getItem('token');
sessionStorage.removeItem('token');
```

Similar to `localStorage`, but tab-specific.

Q48 What are cookies?

Ans: Small pieces of data stored in the browser and sent with every HTTP request to the server.

```
document.cookie = "user=Shubh; expires=" +
  new Date(Date.now()+7*24*60*60*1000).toUTCString();
let c = document.cookie;
```

Sent in HTTP headers; used for auth, sessions, and preferences.

Q49 What is the Fetch API?

Ans: A modern, Promise-based way to make HTTP requests.

```
fetch('https://api.example.com/posts/1')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log('Error:', err));
```

Q50 What is Axios?

Ans: A popular Promise-based HTTP client library that works in both browsers and Node.js.

Feature	Axios	Fetch
JSON parsing	Automatic	Manual (<code>res.json()</code>)
Interceptors	Yes	No
Request cancel	Yes	No (native)

51) What are arrow functions?

Ans: A shorter function syntax that does not have its own this binding — it uses the enclosing scope's this.

```
const add = (a, b) => a + b;
const square = x => x * x;
const sayHi = () => "Hi Shubh";
```

52) What are template literals?

Ans: Allow embedded expressions and multi-line strings using backticks.

```
const name = "Shubh", age = 25;
const msg = `Hi ${name}, you are ${age} years old.`;
```

53) What are modules?

Ans: Modules split code into separate files, exporting and importing parts as needed.

```
// math.js
export const PI = 3.14;
export function add(a,b){ return a+b; }

// app.js
import { PI, add } from './math.js';
```

54) What is destructuring? (recap)

Ans: Unpacking values from arrays or object properties into variables — supports renaming and default values.

```
const { name: n, city = "Delhi" } = { name: "Shubh" };
// n = "Shubh", city = "Delhi"
```

55) What is the spread operator? (recap)

Ans: Expands an iterable into individual elements — arrays, objects, or function arguments.

```
function sum(a,b,c){ return a+b+c; }
const arr = [1,2,3];
sum(...arr); // 6
```

Opposite of the rest operator.

56) What is the rest operator? (recap)

Ans: Collects multiple elements into a single array, used in function parameters and destructuring.

```
function sum(...nums) {
  return nums.reduce((a,b) => a+b, 0);
}
sum(1,2,3,4); // 10
```

57) What is a Set?

Ans: A collection of unique values — duplicates are automatically ignored.

```
const set = new Set();
set.add(1); set.add(2); set.add(2); // ignored
set.has(1); // true
set.size; // 2
```

Methods: add(), has(), delete(), clear(), size.

58) What is a Map?

Ans: A collection of key-value pairs where keys can be of any type.

```
const map = new Map();
map.set('name', 'Shubh');
map.get('name'); // Shubh
map.has('name'); // true
```

Methods: set(), get(), has(), delete(), clear(), size.

59) Promise.all vs Promise.allSettled?

Feature	Promise.all	Promise.allSettled
Resolves with	Array of resolved values	Array of all results (status + value/reason)
Rejects if	Any promise rejects	Never rejects
Use case	All must succeed	Need results of all, success or fail

```
Promise.all([p1, p2]).then(res => ...).catch(err => ...);
Promise.allSettled([p1, p2]).then(res => ...);
```

60) What are generators?

Ans: Special functions that can pause and resume execution using yield. They return an iterator.

```
function* count() {
  yield 1; yield 2; yield 3;
}
const gen = count();
gen.next(); // {value:1, done:false}
gen.next(); // {value:2, done:false}
```

Helpful for lazy evaluation and infinite sequences.

61) == vs ===?

Ans: == compares values after type coercion; === compares both value and type, with no conversion.

```
5 == '5' // true (coerced)
5 === '5' // false (different types)
null == undefined // true
null === undefined // false
```

Use === unless you specifically need type coercion.

62) null vs undefined?

Ans: null is an intentional absence of value; undefined means a variable is declared but not yet assigned.

```
let a; console.log(a); // undefined
let b = null; console.log(b); // null
typeof a; // "undefined"
typeof b; // "object" (historical bug)
```

63) What is NaN?

Ans: "Not-a-Number" — represents an invalid numeric result.

```
0/0; // NaN
isNaN(NaN); // true
Number.isNaN(NaN); // true
```

NaN is the only value in JS that is not equal to itself (NaN !== NaN).

64) What is type coercion?

Ans: Automatic type conversion performed by the JS engine during operations.

```
'5' + 2; // '52' (string concat)
'5' - 2; // 3 (string-number)
true + 1; // 2
```

65) What is execution context?

Ans: The environment in which JS code is executed, with two phases: Creation and Execution.

- **Global Execution Context** → default context
- **Function Execution Context** → created per function call
- **Eval Execution Context** → created by eval()

66) What is garbage collection?

Ans: The process of automatically removing unreachable objects from memory to free space.

Use Chrome DevTools (Memory tab) to detect memory leaks.

67) What is a memory leak?

Ans: Happens when objects are no longer needed but are still referenced somewhere, so they can't be garbage collected.

- Global variables
- Unremoved event listeners
- Closures holding references
- setInterval not cleared

68) Microtask queue vs macrotask queue?

Feature	Microtask	Macrotask
Examples	Promise.then, queueMicrotask, MutationObserver	setTimeout, setInterval, I/O, DOM events
Priority	Higher	Lower
Execution	Before next macrotask	After microtasks are cleared

69) Promise.all vs Promise.allSettled? (recap)

See Q59 for full comparison table — both run promises in parallel but differ in rejection behavior.

70) Promise.allSettled vs any vs race?

Method	Resolves with	Rejects if	Use case
all()	All values	Any rejects	Wait for all to succeed
allSettled()	All results (status+value)	Never	Get all results
any()	First fulfilled value	All reject	Need 1 success
race()	First settled (resolve/reject)	First reject	Whoever finishes first

```
Promise.any([p1, p2, p3]); // resolves with first success
Promise.race([p1, p2, p3]); // settles with first result
```

Remember: understand the concept, not just the answer. Code daily. Think. Debug. Repeat. You've got this!